COMPLEXITY-DEEP: Deterministic Lexical Residual Routing for Language Model MLPs

Anonymous authors
Paper under double-blind review

Abstract

We present COMPLEXITY-DEEP, a decoder-only language model that augments a shared dense MLP backbone with a small deterministic lexical residual branch. A permuted-modulo primary assignment and a deterministic balanced secondary assignment select two experts per token; routing therefore requires neither a learned router nor an auxiliary load-balancing loss. We evaluate the design in a matched single-seed comparison at approximately 300M parameters over 8B FineWeb-Edu tokens. On the fixed evaluation stream drawn from the FineWeb-Edu training split, at the last common evaluation checkpoint (step 7,500), the Token-Routed model reaches loss 2.932897 versus 2.948246 for the dense baseline. At step 7,620, train loss is 2.923100 versus 2.932437, and the mean over the last 50 unique logged steps is 2.928258 versus 2.944597. These results support a deliberately narrow claim: in this matched run pair, a small deterministic lexical branch improves a shared dense backbone at equal token budget. They do not establish a wall-clock efficiency advantage, an independent held-out validation result, or frequency-aware routing, and replication across seeds remains future work.

1 Introduction

Large Language Models (LLMs) have revolutionized natural language processing in recent years (Vaswani et al., 2017; Brown et al., 2020). However, dominant architectures present several limitations:

- **Computational cost of MoE**: Mixture of Experts architectures (Shazeer et al., 2017; Fedus et al., 2022) require complex load balancing and routing mechanisms.
- **Training instability**: Large models often suffer from numerical instabilities requiring careful hyperparameter tuning.

We study COMPLEXITY-DEEP, an architecture centered on deterministic lexical residual routing:

1. **Fixed lexical routing**: Token IDs are mapped to expert pairs by deterministic, layer-specific permutations of modulo partitions, eliminating learned routing and auxiliary load-balancing losses.
2. **Shared-plus-routed residual MLP**: A large dense MLP processes every token while small routed experts provide token-identity-conditioned residual capacity.

The routed branch should be viewed as a *residual lexical path* on top of a shared dense MLP, not as a wholesale replacement for dense computation. The shared branch supplies context-dependent capacity available to every token, while the routed branch adds a small amount of token-identity-conditioned capacity. Our evidence concerns the combined design; it does not by itself establish semantic expert specialization.

2 Related Work

2.1 Transformer Architectures

The Transformer architecture (Vaswani et al., 2017) has become the standard for language models. Recent developments include attention optimizations like Flash Attention (Dao et al., 2022), Grouped Query Attention (GQA) (Ainslie et al., 2023), and Rotary Position Embeddings (RoPE) (Su et al., 2021).

2.2 Mixture of Experts

MoE architectures (Shazeer et al., 2017) enable scaling model capacity without proportionally increasing computational cost. Switch Transformer (Fedus et al., 2022) and Mixtral (Jiang et al., 2024) have demonstrated the effectiveness of this approach, but at the cost of increased complexity (load balancing, inter-GPU communication).

2.3 Fixed Lexical Routing and Shared Experts

The closest precedent for routing by token identity is Hash Layers (Roller et al., 2021), which selects feed-forward parameters through fixed hashes and reports that local token features can compete with learned routing without router parameters or auxiliary balancing losses. COMPLEXITY-DEEP differs by retaining a large dense path for every token and using the fixed lookup only for a narrow top-2 residual branch. Shared experts have also been used in DeepSeekMoE to capture common knowledge while reducing redundancy among routed experts (Dai et al., 2024). Recent statistical analysis gives conditional sample-efficiency results for shared-expert mixtures under explicit estimation assumptions (Nguyen et al., 2025). Those guarantees do not directly apply here because our expert selection is a fixed lexical lookup and our evidence is a single matched run pair; we cite them as motivation for the shared-plus-routed decomposition, not as a theorem about the evaluated checkpoint.

2.4 Normalization and Stability

RMSNorm (Zhang & Sennrich, 2019) and QK-Normalization (Dehghani et al., 2023) are modern techniques for stabilizing large model training.

3 COMPLEXITY-DEEP Architecture

3.1 Overview

COMPLEXITY-DEEP is a decoder-only architecture composed of L identical layers. Each layer comprises:

1. A multi-head attention block
2. An MLP block with token routing
3. Residual connections with pre-normalization (RMSNorm)

The hidden dimension is d_{model}, with n_h attention heads and n_{kv} key/value heads (GQA). Figure 1 provides a simplified schematic of the 300M residual Token-Routed design used in the corrected scaling comparison.

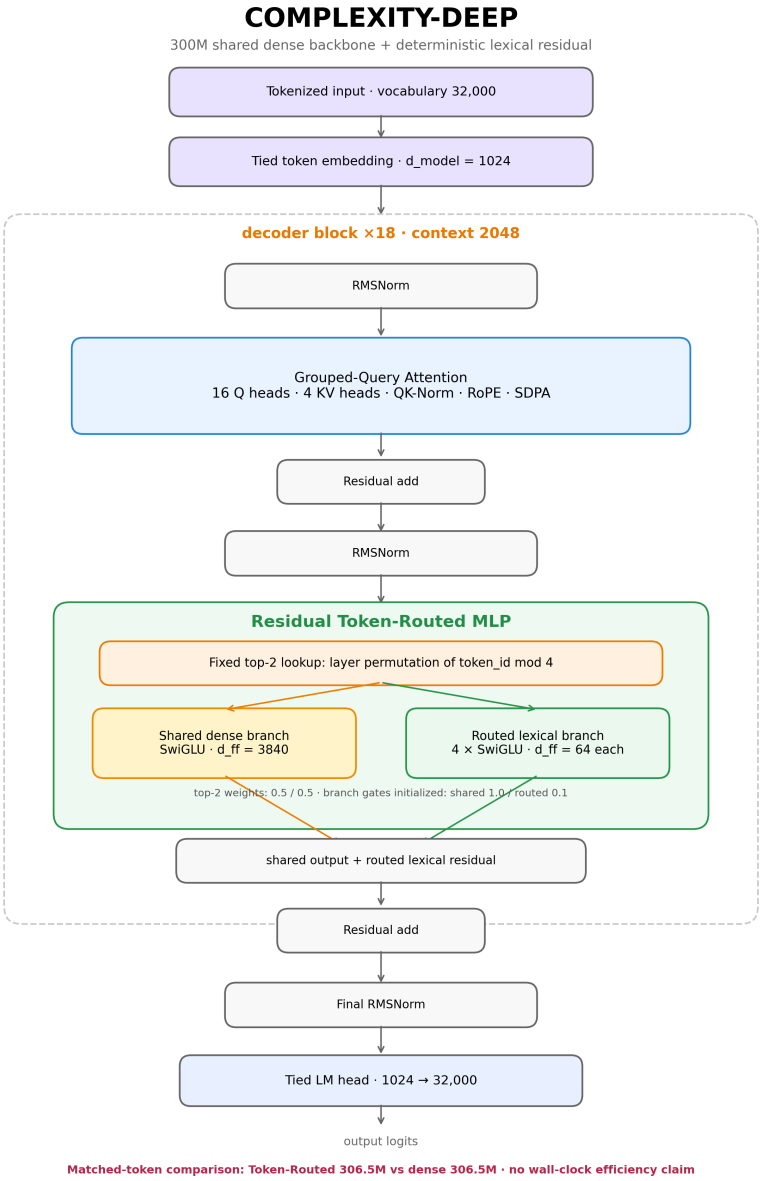

Figure 1: Simplified schematic of the 300M COMPLEXITY-DEEP architecture. Each decoder layer comprises GQA attention followed by a shared dense MLP branch plus a small deterministically routed lexical residual branch.

3.2 Token-Routed MLP

Unlike learned MoEs, the evaluated model uses a fixed top-2 lookup table determined solely by token identity. For layer l, the primary expert is a seeded permutation π_l of the token-ID residue:

$$r_{l,1}(t) = \pi_l(t \bmod E), \qquad E = 4. \tag{1}$$

The 32,000-entry vocabulary is therefore divided exactly evenly: every layer assigns 8,000 token IDs to each primary expert. The secondary table is constructed once, in ascending token-ID order, by selecting the currently least-used expert other than the primary:

$$r_{l,2}(t) = \arg\min_{e \neq r_{l,1}(t)} \sum_{u=0}^{t-1} \mathbb{1}[r_{l,2}(u) = e], \tag{2}$$

with deterministic lowest-index tie breaking. This is the *modulo-primary/balanced-secondary* rule realised by the evaluated checkpoints. Both routed outputs receive fixed weight 0.5 before the learned routed-branch scalar is applied.

For hidden state $\mathbf{x}$ associated with token ID t, the evaluated residual MLP is

$$\mathrm{MLP}_l(\mathbf{x}, t) = g_l^s \,\mathrm{Shared}_l(\mathbf{x}) + g_l^r \sum_{j=1}^{2} 0.5 \,\mathrm{Expert}_{r_{l,j}(t)}(\mathbf{x}), \tag{3}$$

where g_l^s and g_l^r are learned scalar branch gates, initialized to 1.0 and 0.1, respectively. These branch gates are distinct from the fixed 0.5/0.5 weights used to combine the selected experts. Routing is lexical rather than semantic: contextual hidden states vary with the sequence, but expert selection does not.

Each expert is a standard MLP with SiLU activation (SwiGLU):

$$\mathrm{Expert}_i(\mathbf{x}) = (\mathrm{SiLU}(\mathbf{x}\mathbf{W}^i_{gate}) \odot \mathbf{x}\mathbf{W}^i_{up})\mathbf{W}^i_{down} \tag{4}$$

What lexical routing conditions. Fixed routing does not cluster hidden states by meaning. It assigns each token identity to two expert functions, while those functions receive contextual hidden states produced by preceding attention layers. The language-model loss is computed only after the complete residual network and output head; it is not an independent per-expert objective. Different lexical assignments may induce different contextual training distributions, but disjoint token IDs do not imply independent gradients, orthogonal weights, or semantic specialization. The matched comparison therefore evaluates the combined shared-plus-routed architecture, not a claim about individual expert semantics.

Operational advantages:

- No auxiliary load balancing loss (hyperparameter savings)
- Static expert lookup with no learned expert-selection network

3.3 Final Architecture

The evaluated architecture combines sparse dispatch, a large shared branch, and a small top-2 routed residual branch.

3.3.1 Sparse Routed-Branch Dispatch

The implementation evaluates each routed expert only on the tokens assigned to it:

$$\mathrm{out}[\mathrm{mask}_e] = \mathrm{Expert}_e(\mathbf{x}[\mathrm{mask}_e]) \quad \text{for } e = 0, \ldots, E-1 \tag{5}$$

This avoids evaluating every routed expert for every token. It does not make the complete shared-plus-top-2 model equivalent to a 1× dense MLP in wall-clock cost.

3.3.2 Shared Lexical Expert

A shared dense MLP processes all tokens, while two narrow experts provide a lexical residual. The output combines both branches:

$$\mathrm{MLP}_{\mathrm{output}}(\mathbf{x}, t) = g_s \mathrm{SwiGLU}_{\mathrm{shared}}(\mathbf{x}) + g_r \sum_{j=1}^{2} 0.5\, \mathrm{SwiGLU}_{r_j(t)}(\mathbf{x}) \tag{6}$$

where each component is a standard SwiGLU MLP:

$$\mathrm{SwiGLU}_{\mathrm{shared}}(\mathbf{x}) = (\mathrm{SiLU}(\mathbf{x}\mathbf{W}^s_{gate}) \odot \mathbf{x}\mathbf{W}^s_{up})\mathbf{W}^s_{down} \tag{7}$$

$$\mathrm{SwiGLU}_e(\mathbf{x}) = (\mathrm{SiLU}(\mathbf{x}\mathbf{W}^e_{gate}) \odot \mathbf{x}\mathbf{W}^e_{up})\mathbf{W}^e_{down} \tag{8}$$

In the 300M checkpoint, the shared intermediate width is 3,840, while each of four routed experts has width 64; two routed experts are active per token. Thus the routed component is intentionally a small correction to a dense-compatible shared trunk, not a replacement of equal width.

3.3.3 Vocabulary Assignment

The saved lookup tables confirm layer-specific permuted-modulo routing. Because 32,000 is divisible by four, each primary expert receives exactly 8,000 vocabulary entries. This is vocabulary-cardinality balance, not corpus-frequency balance.

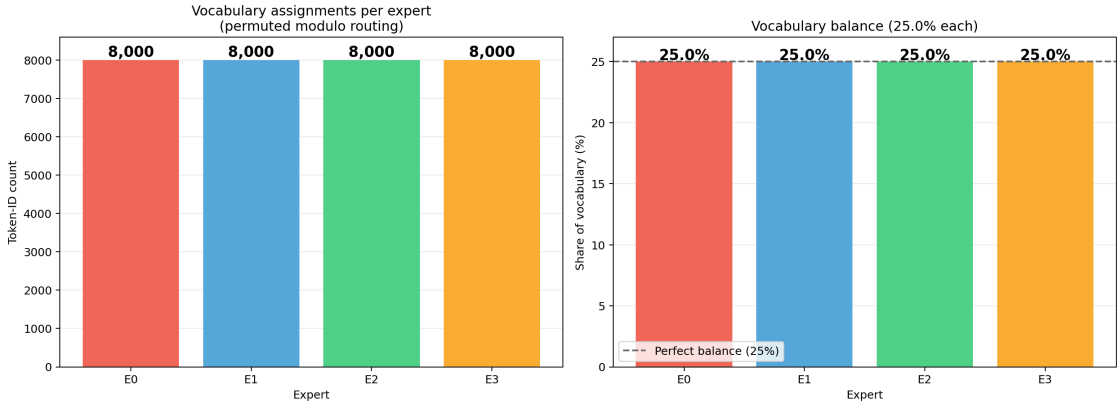

Figure 2: Vocabulary assignment recovered from the 300M checkpoint. Every layer assigns exactly 8,000 of the 32,000 token IDs to each primary expert through a layer-specific permutation of modulo routing. This figure does not represent corpus-frequency traffic.

4 Theoretical Analysis

We state only properties that apply directly to the evaluated shared-plus-top-2 architecture.

4.1 Primary Vocabulary Balance

Proposition 4.1 (Primary assignment balance)**.** *Let token IDs be* $\{0, \ldots, |V|-1\}$ *and let* E *be the number of experts. Under* $r_{l,1}(t) = \pi_l(t \bmod E)$*, each expert receives either* $\lfloor |V|/E \rfloor$ *or* $\lceil |V|/E \rceil$ *primary token IDs. If* $E \mid |V|$*, each receives exactly* $|V|/E$ *IDs.*

Proof. Before permutation, residue class e contains the IDs $e, e+E, e+2E, \ldots$ below $|V|$. Their cardinalities differ by at most one. Because π_l is a bijection of expert labels, it preserves these cardinalities. For $|V| = 32000$ and $E = 4$, every layer therefore assigns exactly 8,000 primary IDs to each expert. □

This is vocabulary-cardinality balance, not corpus-traffic balance. Non-uniform token frequencies can produce unequal realised traffic even when each expert owns the same number of primary IDs.

4.2 Matched Capacity and Active MLP Width

Let d be the model dimension, d_s the shared SwiGLU width, d_e each routed-expert width, E the number of stored experts, and k the number selected per token. Ignoring biases, the MLP parameter count per layer is

$$P_{\mathrm{TR}} = 3d(d_s + Ed_e), \tag{9}$$

while the width evaluated for each token is

$$d_{\mathrm{active}} = d_s + kd_e. \tag{10}$$

For the evaluated model, $(d_s, E, d_e, k) = (3840, 4, 64, 2)$. Hence the stored MLP width is $3840+4\times64 = 4096$, exactly matching the dense baseline width, whereas each token traverses width $3840 + 2 \times 64 = 3968$. This accounting describes the evaluated architecture; it does not imply a wall-clock speedup because dispatch and small expert kernels add overhead. Measured training throughput is reported in Section 7.5.

4.3 Scope of Interpretation

The deterministic lookup removes learned expert selection and auxiliary load-balancing losses, but also gives up content-adaptive routing. Fixed lexical assignments may lead experts to learn different functions, yet the reported measurements do not establish semantic or functional specialization. Our evidence concerns end-to-end language-model loss, routing-table structure, and observed expert traffic.

5 Implementation

5.1 Technical Specifications

Our implementation uses PyTorch 2.0+ with the following optimizations:

Table 1: Implementation choices

Component	**Technical Choice**
Attention	SDPA / Flash Attention
Positional Encoding	RoPE ($\theta = 10000$)
Normalization	RMSNorm + QK-Norm
Activation	SiLU (SwiGLU)
Precision	BF16 (training and inference)

5.2 300M Model Configuration

For the corrected iso-parameter scaling comparison (Section 7.5), we train two architectures at ∼300M parameters (Table 2): a residual Token-Routed model (306.5M, 4 experts, top-k = 2, large shared expert and small routed branch) and a dense SwiGLU baseline (306.5M, $d_{ff} = 4096$). Both share the same backbone (18 layers, $d_{model} = 1024$, GQA 16/4, RMSNorm, QK-Norm), tokenizer, sequence length, optimizer, warmup schedule, and FineWeb-Edu token budget.

6 Experiments

6.1 Pilot Study

Earlier pilot runs motivated removing the adaptive residual controller and retaining a shared dense path. The evidence in this paper is centered on the corrected 300M iso-parameter comparison (Table 2); pilot runs are not used to support the main claim.

Table 2: COMPLEXITY-DEEP 300M model configurations (iso-params comparison)

Parameter	**Token-Routed (MoE)**	**Dense Baseline**
Layers (L)	18	18
Dimension (d_{model})	1024	1024
Attention heads (n_h)	16	16
KV heads (n_{kv})	4 (GQA ratio 4:1)	4 (GQA ratio 4:1)
Routed intermediate dimension	256 total	—
Expert intermediate	64	—
Shared expert intermediate	3840	—
Experts ($n_{experts}$)	4, top-$k = 2$	1 (dense)
Shared/routed gate initialization	1.0 / 0.1	—
Vocabulary	32000	32000
Max context	2048	2048
Total parameters	**306.5M**	**306.5M**
Active MLP path/token	shared + 2 routed experts	dense SwiGLU

6.2 Training Recipe

Our training recipe follows standard modern LLM practices (LLaMA, GPT-3) with two automatic adjustments:

Dynamic warmup. Instead of a fixed number of warmup steps (which can represent 52% of training for short runs), warmup is automatically set to **5% of total steps**. This ensures a constant warmup/training ratio regardless of run duration.

GPT-style initialization. Residual output projections (`o_proj`, `down_proj`) are initialized with reduced standard deviation:

$$\sigma_{\text{residual}} = \frac{0.02}{\sqrt{2 \times L}} \tag{11}$$

where L is the number of layers. This prevents the residual stream from growing with depth (Radford et al., 2019).

Other hyperparameters. Weight decay = 0.1 (standard GPT-3/LLaMA), AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.95$, gradient clipping = 1.0, BF16 precision, cosine scheduler with min_lr = 10% of peak.

6.3 Routing-Table Verification

We audited the final checkpoint rather than inferring routing from run names. For every one of the 18 layers, the saved 32,000-entry primary-routing table is exactly a layer-specific permutation of t mod 4. Each expert therefore receives 8,000 primary vocabulary assignments. The run did not contain a corpus-frequency table, so we make no frequency-aware or corpus-balanced routing claim. The observed expert traffic reported below is an empirical property of this run, not a guarantee of the routing rule.

7 Discussion

7.1 Comparison with Existing Architectures

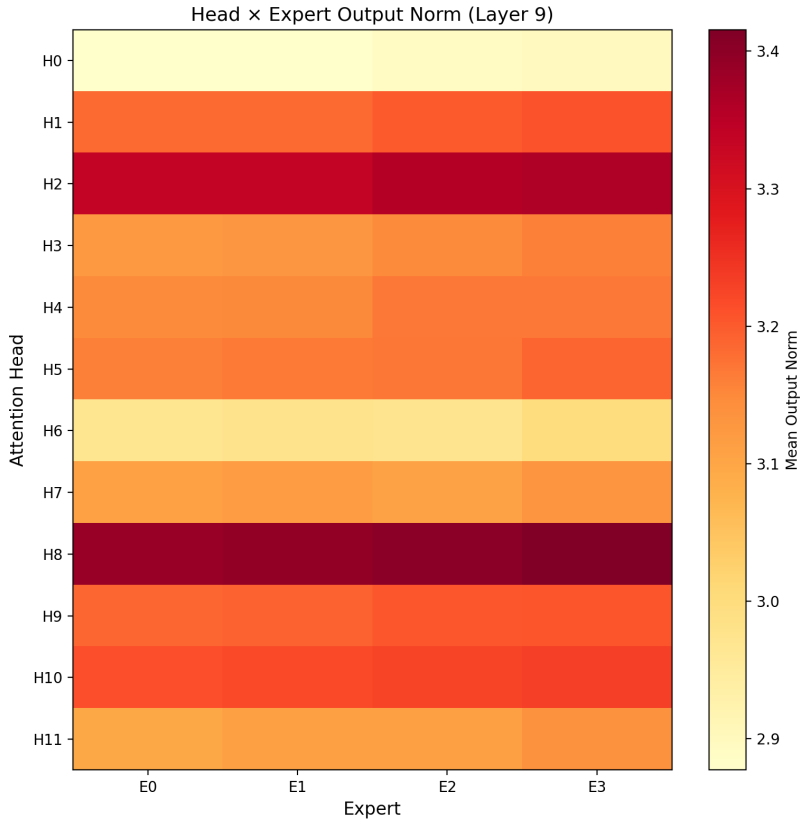

Figure 3: Attention head × expert diagnostic heatmap for the median layer. The figure is descriptive; no head–expert coupling claim is inferred from it.

Table 3: Architectural comparison

Architecture	**Routing**	**Load balancing loss**
Transformer	No	N/A
Switch Transformer	Soft	Required
Mixtral	Top-2	Required
COMPLEXITY-DEEP	**Hard lexical**	**Not used**

7.2 Token-Routing Analysis

The saved routing tables guarantee equal primary vocabulary cardinality, while telemetry measures realised corpus traffic:

- **Near-balanced utilization**: In the 300M run, final observed expert utilization is 0.2481/0.2635/0.2481/0.2403, with no dead experts.
- **Balanced norms**: Expert norms remain close across routed branches.
- **No dead branch**: All four experts receive nonzero observed traffic

These diagnostics rule out dead routed branches in the observed run, but they do not establish semantic or functional specialization of individual experts. We therefore restrict the empirical claim to the end-to-end loss of the combined architecture.

Table 4: Token-Routed MLP vs soft-routing MoE

Criterion	**Token-Routed**	**Standard MoE**	**Advantage**
Expert assignment	Fixed lexical table	Learned gating	Depends
Specialization claim	Not established here	Content-conditioned	Standard MoE
Deployment	No gating network	Gating + dispatch	Token-Routed
Determinism	100%	No (softmax)	Token-Routed
Shared Expert	Yes (common patterns)	Optional	Token-Routed
Selection signal	Fixed token identity	Hidden-state content	Standard MoE

7.3 Dense Computational Substrate for Conditional Modules

The evaluated architecture can be viewed as an instance of a broader dense-first integration pattern. Let F_{dense} denote a transformation applied uniformly to every token and $G_{\text{conditional}}$ a structured module whose parameters or execution path depend on a discrete condition z:

$$\mathbf{y} = F_{\text{dense}}(\mathbf{x}; \theta_s) + \alpha\, G_{\text{conditional}}(\mathbf{x}, z; \theta_c). \tag{12}$$

The dense path provides a regular batched computation expressible through standard matrix-multiplication kernels. The conditional path can remain a narrow residual, limiting the fraction of the layer affected by irregular dispatch, variable group sizes, and small expert kernels. In COMPLEXITY-DEEP, F_{dense} is the shared SwiGLU branch, z is token identity, and $G_{\text{conditional}}$ is the deterministic top-2 lexical residual.

This decomposition does not make arbitrary Python objects directly executable by PyTorch. A conditional module must still be represented using tensors, registered parameters, and differentiable tensor operations; efficient execution may additionally require grouped matrix multiplication or specialized kernels. The shared path instead preserves a large regular computational substrate while the conditional component remains narrow and structured. Our experiments validate only the lexical-routing instance of this pattern. They do not establish that arbitrary conditional modules benefit from the same decomposition or that it improves wall-clock efficiency; the lower measured throughput of the current routed implementation shows that sparse-dispatch overhead remains material.

7.4 Exploratory 100M Ablations (1B tokens)

We retain the requested seven-way 100M suite as an exploratory diagnostic, but correct its labels and scope. Every run uses the same 2×B200 budget: 954 steps × 2 GPUs × batch/GPU 256 × sequence length 2048 = 1.0003B tokens. The FineWeb streaming path did not populate the frequency table expected by the configuration formerly labelled "Zipf," so that condition used modulo-based primary routing with a deterministic alternate secondary route. We call it *modulo-primary/balanced-secondary top-2* below. Nominal modulo and round-robin controls have equivalent primary assignments in this setting; their single-run differences must not be interpreted as causal routing-strategy effects.

The modulo-primary/balanced-secondary shared run has the lowest logged losses in the suite: train loss 4.8205 at step 950 and evaluation loss 4.8887 at step 750, versus 4.8244 and 4.8958 for the dense residual control. These gaps are small, single-seed, and comparable to differences between nominally equivalent controls. The suite therefore provides exploratory evidence only: the no-shared and random-assignment runs are worse in this suite, and the current routed implementation is slower than dense. It does not establish causal routing-strategy effects or validate frequency-aware bin-packing.

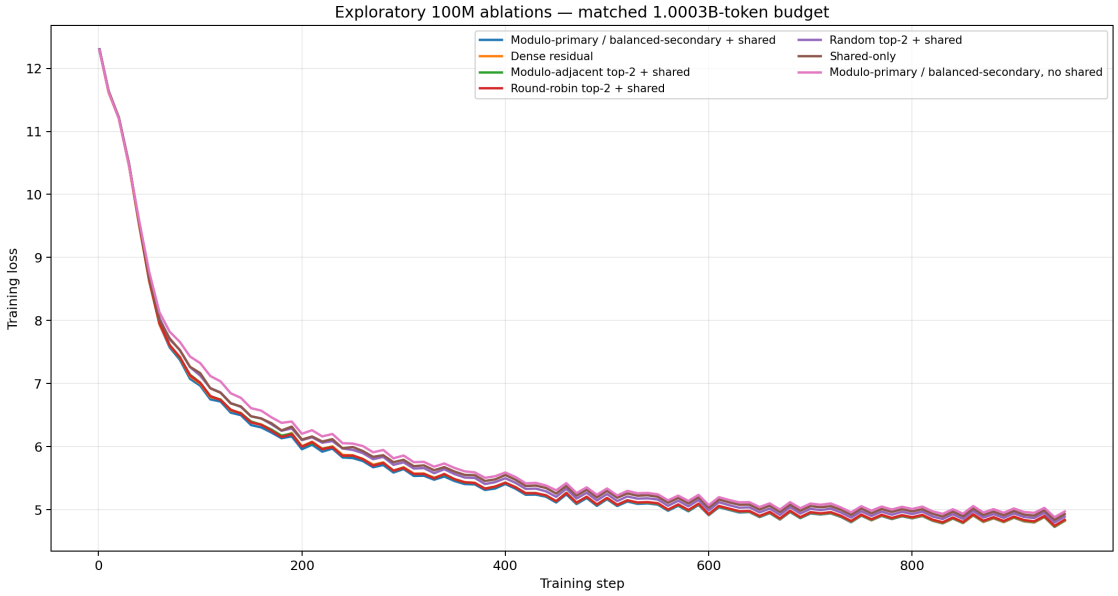

Figure 4: Exploratory 100M training curves at a matched 1.0003B-token budget. The condition originally logged as "Zipf shared" is identified by its realised fixed lookup: modulo-primary/balanced-secondary top-2. Single-run differences between modulo and round-robin controls are not interpreted causally.

Table 5: Exploratory single-seed 100M runs on 2×B200. All rows use 1.0003B tokens; evaluation values on the fixed FineWeb-Edu training-split stream are at step 750.

Variant	Train loss @ 950	Val. loss @ 750	Tok/s near end
Modulo-primary/balanced-secondary + shared	**4.8205**	**4.8887**	321k
Dense residual	4.8244	4.8958	397k
Modulo-adjacent top-2 + shared	4.8320	4.9016	321k
Round-robin top-2 + shared	4.8384	4.9072	321k
Random top-2 + shared	4.8875	4.9577	318k
Shared-only	4.9285	5.0012	**402k**
Modulo-primary/balanced-secondary, no shared	4.9693	5.0393	334k

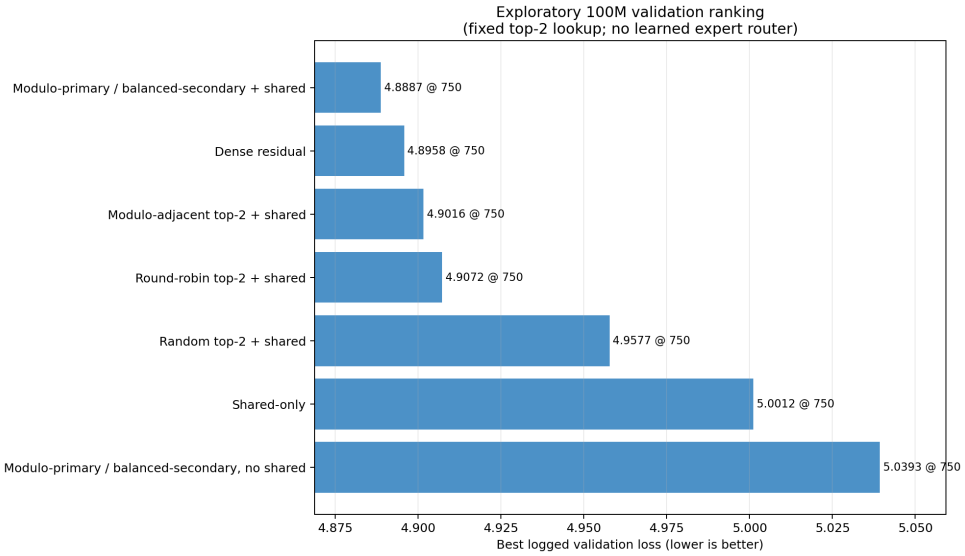

Figure 5: Evaluation losses for the exploratory 100M suite. All routed variants use fixed top-2 lookup tables; none uses a learned expert router.

7.5 Scaling Comparison (300M, 8B tokens)

To validate the corrected architecture at larger scale, we train iso-parameter models (Table 2) on an 8B-token FineWeb-Edu budget. Both runs use the same global batch of 1,048,576 tokens/step (8 GPUs $\times$ batch 64 $\times$ sequence length 2048), so matched raw step number equals matched tokens seen, and no token-grid interpolation is required.

The two runs start from comparable random-init loss (TR 10.58, dense 10.54). The Token-Routed model places most MLP capacity in the shared dense path and uses a much smaller fixed lexical branch as a residual. It trails the dense baseline early: at step 100 ($\approx$105M tokens) the gap is $+0.177$ in favor of dense, peaking near $+0.31$ around step 40. The first logged train-loss win appears at step 740 ($\approx$776M tokens), and the first matched evaluation-stream win appears at step 750. At the last common evaluation checkpoint (step 7,500; 7.864B tokens), Token-Routed reaches **2.932897** versus **2.948246** for dense, a gap of -0.015349. At step 7,620 ($\approx$7.99B tokens), train loss is **2.923100** versus **2.932437**; after deduplicating resumed log rows by step, the mean over the last 50 unique logged steps is **2.928258** versus **2.944597**, a gap of -0.016339. Observed traffic remains near-balanced (0.2481/0.2635/0.2481/0.2403, zero dead experts), although the routing rule itself guarantees only equal vocabulary cardinality. This evaluation stream is drawn from the FineWeb-Edu training split and is not an independent held-out validation set.

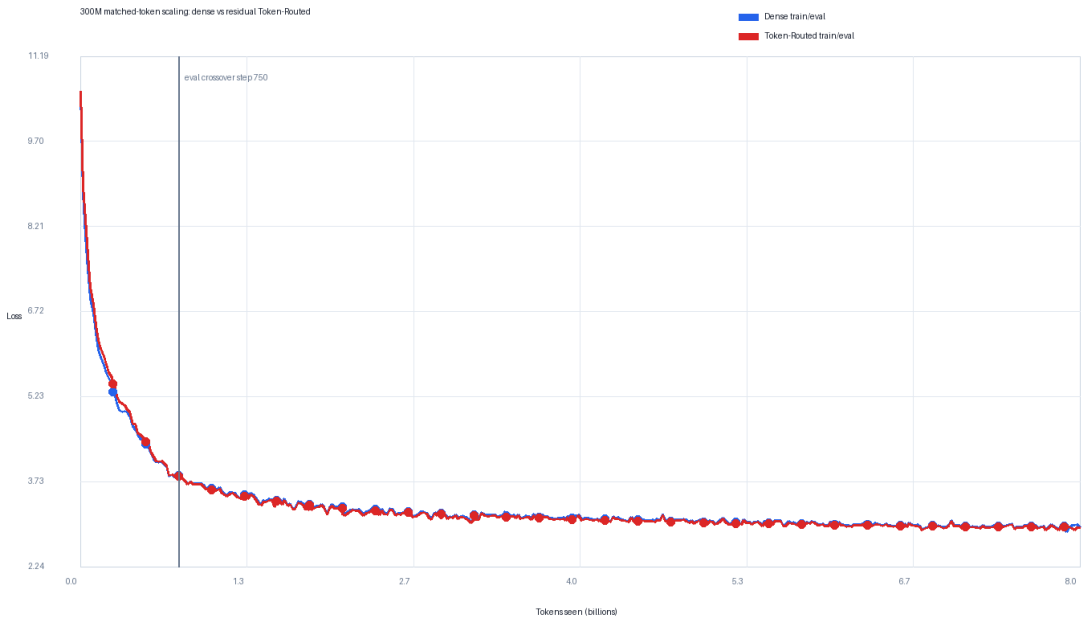

Figure 6: Corrected 300M scaling curves at iso-batch (1,048,576 tokens/step). Token-Routed (red) trails dense (blue) during the first ≈700 steps, reaches parity, then crosses over: by step 1000 (≈1.05B tokens) Token-Routed leads.

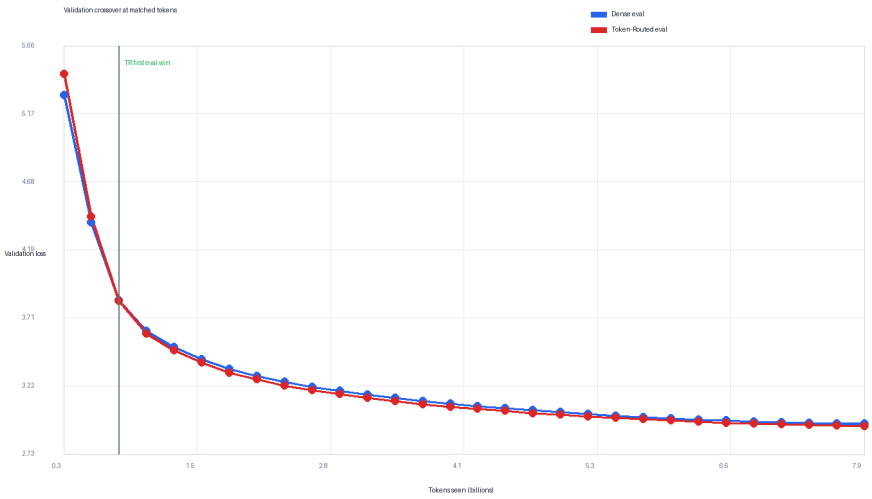

Figure 7: Matched evaluation-stream loss. Token-Routed has an early deficit at the first two evaluation points, crosses the dense baseline at step 750 (≈786M tokens), and remains ahead through the last common evaluation checkpoint at step 7,500 (7.864B tokens). The stream is drawn from the FineWeb-Edu training split, not a held-out validation split.

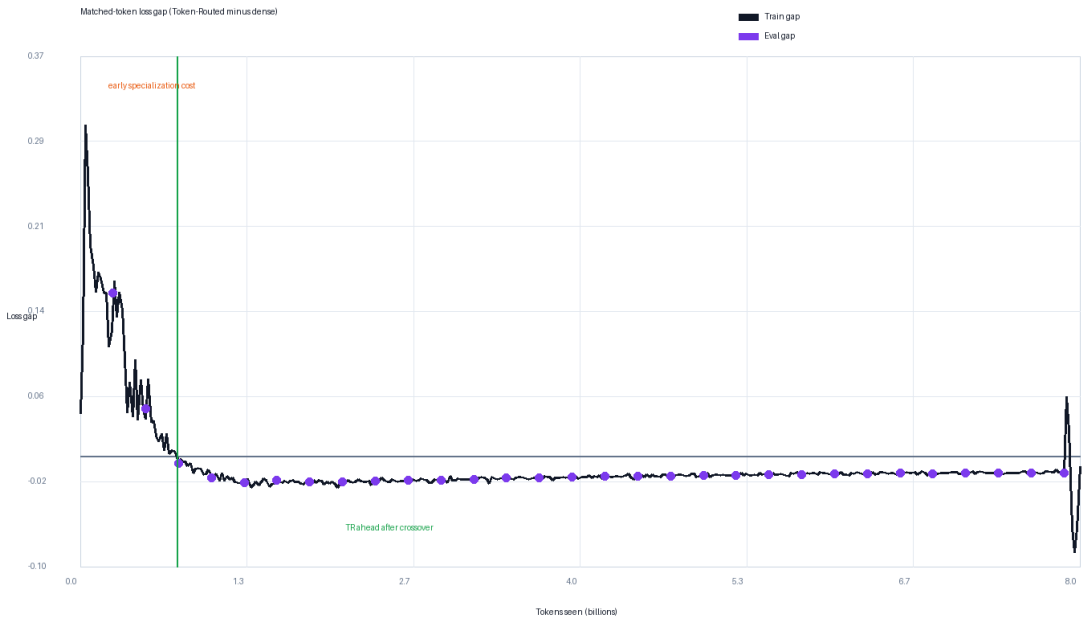

Figure 8: Training-loss gap at matched step (= matched tokens, iso-batch), computed as Token-Routed loss minus dense loss. Negative values indicate Token-Routed ahead. The gap is positive during the early transient and first turns negative at logged step 740.

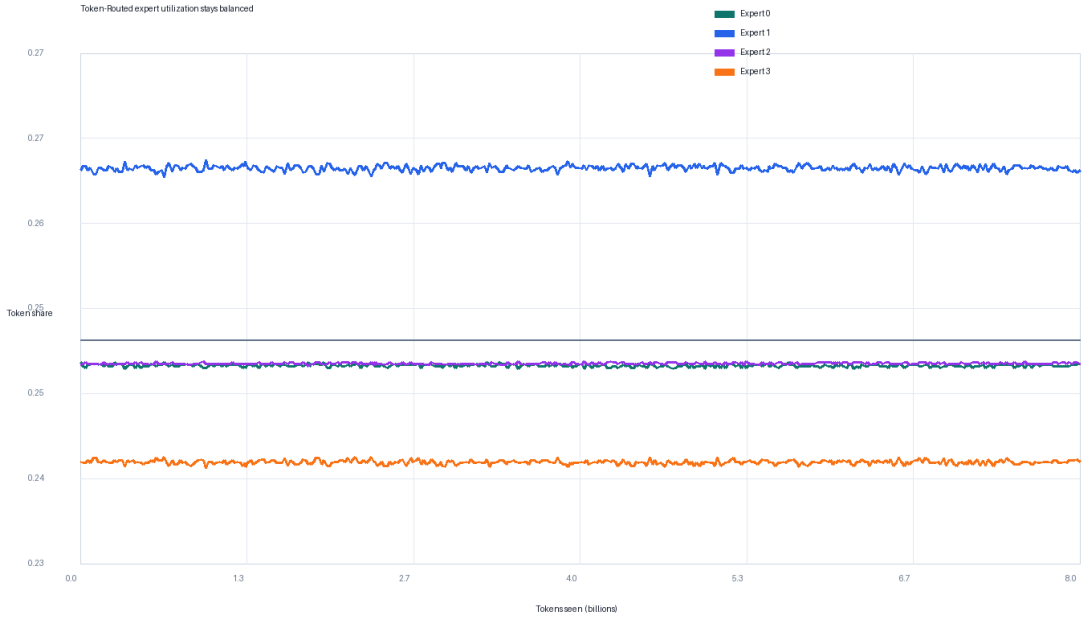

Figure 9: Expert utilization during the corrected 300M Token-Routed run. All four experts remain active and near-balanced; the observed matched-budget win is not caused by expert collapse.

Table 6: 300M scaling comparison at iso-batch (1,048,576 tokens/step), full 8B-token run. Train loss is reported at matched step. Negative gap indicates Token-Routed ahead. Last row is the smoothed mean of the final 50 logged steps.

Step	**Tokens seen**	**Dense (306.5M)**	**TR (306.5M, k=2)**	**Gap (TR − Dense)**
100	105M	6.6698	6.8464	+0.177
500	524M	4.4450	4.4796	+0.035
700	734M	3.8567	3.8619	+0.005
1000	1.05B	3.5500	**3.5324**	**−0.018**
2000	2.10B	3.1953	**3.1720**	**−0.023**
4000	4.19B	3.0925	**3.0738**	**−0.019**
6000	6.29B	3.0061	**2.9906**	**−0.015**
7620	7.99B	2.9324	**2.9231**	**−0.009**
last 50 (smoothed)	—	2.9446	**2.9283**	**−0.016**

Training throughput. The 300M Token-Routed model uses a large shared branch plus a small top-$k = 2$ routed branch, so it is not intended as a pure top-1 speed benchmark. Both runs were trained on 8×B300, dense reaching approximately 0.95M tokens/s and Token-Routed approximately 0.75M tokens/s at identical global batch (1,048,576 tokens/step). All quality comparisons above are at matched tokens seen. A matched-wall-clock comparison would answer a different question—whether the current implementation is more compute-efficient—and would favor the faster dense baseline unless the routed residual branch is further optimized. We therefore report throughput separately and do not claim wall-clock efficiency. The routing table lookup itself is $O(1)$ and does not require learned-router synchronization.

7.6 Inference Sanity Check

To check that deterministic lexical routing is compatible with standard serving stacks, we also benchmarked an earlier small Token-Routed checkpoint on vLLM 0.18 with PagedAttention and CUDA graphs. This serving result is not used as evidence for the corrected 300M scaling comparison above, and it should not be interpreted as a compute-efficiency claim for the 300M checkpoint. It only verifies that deterministic per-token routing can be deployed without a learned router or all-to-all dispatch.

On a single NVIDIA RTX PRO 6000 GPU (96 GB), serving 100 concurrent requests at 10 RPS with 128 input tokens and 1,900 output tokens, the checkpoint reaches a sustained throughput of **8,078 tokens/s** (peak 10,179 tokens/s), with median time-to-first-token (TTFT) of 29.3 ms and median inter-token latency (ITL) of 7.9 ms (Figure 10). Updated inference measurements for the corrected 300M checkpoint remain future work.

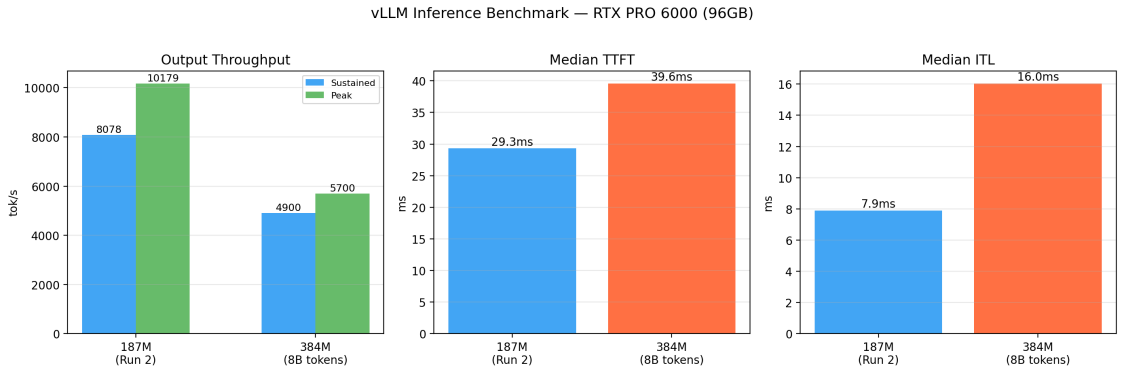

Figure 10: Inference sanity check on a single NVIDIA RTX PRO 6000 (96 GB). 100 requests at 10 RPS, 128 input tokens, 1,900 output tokens. Sustained throughput: 8,078 tok/s; peak: 10,179 tok/s; median TTFT: 29.3 ms; median ITL: 7.9 ms. This benchmark uses an earlier small checkpoint and is reported only as a deployment compatibility check.

7.7 Limitations and Future Work

- **Single-seed scaling result**: The corrected 300M run is a full 8B-token matched-budget comparison, but it is still a single seed per architecture. Additional seeds would quantify whether the final -0.0163 smoothed gap is robust to initialization variance.

- **No frequency-aware routing result**: The evaluated checkpoints use permuted-modulo lexical tables. Frequency-aware bin-packing exists as an unevaluated code path and is not supported by the reported results.

- **Frequency-aware routing as future work**: A clean follow-up should estimate token frequencies from the exact training stream, construct and serialize the greedy Zipf bin-packing table before model initialization, verify expected and realised expert traffic, and compare it against the verified permuted-modulo baseline over multiple seeds. Checkpoints should record a routing-table hash and frequency-source metadata so that silent fallback is impossible.

- **Standardized benchmarks**: Zero-shot evaluation on ARC-Easy, HellaSwag, and MMLU via `lm-evaluation-harness` should be rerun on the corrected 300M checkpoints.

8 Conclusion

We presented COMPLEXITY-DEEP, a language model architecture that augments a shared dense MLP backbone with a small deterministic lexical residual branch, without a learned router or auxiliary load-balancing loss.

In the corrected iso-parameter 300M scaling run at iso-batch (1,048,576 tokens/step, approximately 8B tokens), the residual Token-Routed variant has an early deficit during the first $\approx$700 steps (peak gap $+0.31$ near step 40), first wins on logged train loss at step 740 and on the fixed evaluation stream at step 750. At the last common evaluation checkpoint (step 7,500), it remains ahead; the final train-loss gap over the last 50 unique logged steps is -0.0163 in favor of Token-Routed (2.9283 vs 2.9446). These results support deterministic lexical routing as a promising matched-token quality signal when used as a residual lexical branch on top of a shared dense MLP backbone. They should not be read as evidence that routing alone replaces dense MLP computation, that the evaluation is held out, or that the result generalizes across seeds. Multi-seed scaling and compute-normalized comparisons remain important next steps.

Broader Impact Statement

This work studies an architectural modification for language models. The submission does not include model weights; any future release should follow the applicable data, safety, and licensing requirements.

Reproducibility Statement

For review, the supplementary archive provides an audit-corrected PyTorch implementation, exact routing tests, run configurations, figure-generation scripts, and the verified 32k tokenizer. It is a reproduction artifact rather than an untouched historical source snapshot; the reported limitations identify measurements not included in the archive.

References

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pp. 1877–1901, 2020.

Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.

Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pp. 16344–16359, 2022.

Mostafa Dehghani, Josip Djolonga, Basil Mustafa, Piotr Padlewski, Jonathan Heek, Justin Gilmer, Andreas Peter Steiner, Mathilde Caron, Robert Geirhos, Ibrahim Alabdulmohsin, et al. Scaling vision transformers to 22 billion parameters. In *International Conference on Machine Learning*, pp. 7480–7512, 2023.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *The Journal of Machine Learning Research*, 23(1):5232–5270, 2022.

Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

Huy Nguyen, Thong T. Doan, Quang Pham, Nghi D. Q. Bui, Nhat Ho, and Alessandro Rinaldo. On deepseekmoe: Statistical benefits of shared experts and normalized sigmoid gating. *arXiv preprint arXiv:2505.10860*, 2025.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019.

Stephen Roller, Sainbayar Sukhbaatar, Arthur Szlam, and Jason Weston. Hash layers for large sparse models. *Advances in Neural Information Processing Systems*, 34:17555–17566, 2021.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

A Training Curves

Additional training curves and analysis figures are available in the supplementary material.